# Drawing Alternative Splicing Graphs

Yaw-Ling Lin*
Dept. Computer Science and Information Management
Providence University, Taiwan
E-mail: yllin@pu.edu.tw

## Abstract

*Alternative splicing of a single pre-mRNA can give rise to different mRNA transcripts. Alternative splicing of premessenger RNA is an important layer of gene expression regulation in eukaryotic cell. Consequently, alternative splicing is an important mechanism for generating protein diversity from a single gene. Due to all these important aspects of genomic alternative splicing mechanism that produces different mRNA's, biologists require means of analyzing and visualizations of the alternative splicing forms. In this paper, we consider ways of drawing a given alternative splicing graph on the plane.*

*Given an alternative splicing graph $G = (V, E)$, here in this paper we show that computing the crossing number of a given upper drawing ASG can be done in $O(|E| \log |E|)$ time. The optimal (fewest crossing) drawing of alternative splicing graphs is related to the long known difficult problem of crossing number. Here we propose an efficient branch-and-bound recursive algorithm with heuristic searching strategy. Some experimental results are also discussed in the paper.*

**Keywords:** *algorithm, computational biology, branch and bound, crossing number.*

## 1 Introduction

RNA splicing is an essential, precisely regulated post-transcriptional process that occurs prior to mRNA translation. A gene is first transcribed into a pre-messenger RNA (pre-mRNA), which is a copy of the genomic DNA containing intronic regions destined to be removed during pre-mRNA processing (RNA splicing), as well as exonic sequences that are retained within the mature mRNA. Alternative splicing of eukaryotic pre-mRNAs is a mechanism for generating potentially many transcript isoforms from a single gene. It serves versatile regulatory functions in controlling major developmental decisions and fine-tuning of gene function [9]. The majority of alternative splicing events give rise to different protein products. At least 35% of human genes undergo alternative splicing [6, 5]. The physiological functions of these splice variants may be similar, opposite, or unrelated. In addition, they may differ in other properties, such as stability, tissue and cellular localization, temporal expression pattern, and responses to agonists or antagonists. The presence or level of specific splice variants may cause and/or indicate pathological or normal conditions. A class example is the Prostate Specific Anitgene, the most important maker available today for diagnosing and monitoring patients with prostate cancer [4].

*Exons* are areas of genomes that can be translated to produce mRNA and finally transcribed to proteins. Different combinations of exons can be spliced together to produce different mRNA isoforms of a gene, encoding structurally and functionally different protein products [10]. Note that an *alternative splicing form* (ASF) is composed by linking several alternative splicing sites. In other words, an alternative splicing form is just a list of exons, altogether constitute an expressed mRNA. Given a (hamiltonian) directed acyclic graph $G = (V, E)$ with a source vertex $s \in V$, where the vertex set $V$ represents exons and $E$ represents the (possible alternative splicing) transition between exons. It follows that a (connected) path starting from $s$ is a putative RNA transcript for a given gene of interest. The *splicing graph* $G$ is the directed graph on the set of transcribed sites $V$ that contains a directed edge $(u, v)$ if and only if $u$ and $v$ are consecutive positions in one of the transcripts. Every transcript can be viewed as a path in the splicing graph $G$, and the whole graph $G$ is the union of all paths.

*This work is supported in part by the National Science Council, Taiwan, R.O.C, grant NSC-92-2213-E-126-011.

**Definition 1 (Alternative Splicing Graph)** *A (hamiltonian) directed acyclic splicing graph $G = (V, E)$ is an* alternative splicing graph *(ASG) if $G$ has a single starting (source) vertex $s \in V$, where the vertex set $V$ represents exons and $E$ represents the (alternative splicing) transition between exons.*

Due to all these important aspects of genomic alternative splicing mechanism that produces different mRNA's, biologists require means of analyzing and visualizations of the alternative splicing forms. In this paper, we consider ways of drawing (embedding) a given alternative splicing graph on the plane.

Since an exon is just a consecutive sequence of the genomic sequence, the (5' to 3') genomic positions thus constitute a natural linear ordering. It follows that the drawing of a alternative splicing graph usually places the vertices (exons) on a horizontal line from left to right. Furthermore, the transcribing transition edges are usually drawn as arcs connecting the corresponding exons. One way of embedding these transition edges is to draw them at the upper plane above the horizontal line. The corresponding embedding is called an *(upper drawing) alternative splicing graph* (ud-ASG). However, allowing only upper embedding often leads to too many crossing drawn arcs on the upper plane. The number of crossing arcs can be reduced dramatically by allowing some of these arcs being drawn on the lower plane; see Figure 1 for an illustration of a ud-ASG, and its optimal (fewest crossing number) two-way embedding.

### Crossing Number

Given a graph $G = (V, E)$, the crossing number is the minimum possible number of crossings with which the graph can be drawn. The crossing number of $G$ is usually denoted by $\nu(G)$, that attracts many researchers' interests for many years citeliebers-planarizing. A graph with crossing number 0 is a planar graph. Garey and Johnson [2] showed that determining the crossing number is an NP-complete problem.

In 1954 Zarankiewicz provided what he believed was a proof for the crossing number of the complete bipartite graph [11], where he claimed that $\nu(K_{n,m}) = \lfloor \frac{n}{2} \rfloor \lfloor \frac{n-1}{2} \rfloor \lfloor \frac{m}{2} \rfloor \lfloor \frac{m-1}{2} \rfloor$. The reasoning behind the conjecture is as follows. Place $\lfloor \frac{n}{2} \rfloor$ of the vertices on the positive $x$ axis and $\lceil \frac{n}{2} \rceil$ on the negative x axis; similarly place $\lfloor \frac{m}{2} \rfloor$ of the vertices on the positive and $\lceil \frac{m}{2} \rceil$ on the negative y axis. Connect the vertices on the x axis with those on the y axis by drawing $nm$ straight line edges. Hence this is a drawing on $K_{n,m}$, and the number of crossings can be determined to be exactly as stated above. However, in 1969 R. K. Guy found an error in Zarankiewicz's proof [3]. Hence Zarankiewicz's calculation is not the exact crossing number for the complete bipartite graphs, but it is at least an upper bound to the exact value. In 1971, Kleitman [7], proved that Zarankiewicz's conjecture predicts the correct values for all complete bipartite graphs $K_{5,m}, K_{6,m}$ for all positive $m$.

Guy [3] initiated the search for the crossing number of the complete graphs offering a conjecture as to what the number might be. Guy conjectured that $\nu(K_n) = \frac{1}{4} \lfloor \frac{n}{2} \rfloor \lfloor \frac{n-1}{2} \rfloor \lfloor \frac{n-2}{2} \rfloor \lfloor \frac{n-3}{2} \rfloor$. The construction of this conjecture is a lot more involved and subtle that Zarankiewicz's. To summarize it, the vertices of $K_n$ are mapped to top and bottom discs of a cylinder isomorphic to the unit sphere. Then these vertices are connected to form two complete graphs isomorphic to $K_{n/2}$ on the top and bottom discs. By considering the shortest helical paths on the cylinder between the vertices on the top and bottom discs, the upper bound for the crossing number of complete graph is then obtained.

Today Guy's conjecture on the crossing number of the complete graphs and Zarankiewicz's conjecture for complete bipartite graphs have neither been proven nor disproven. However, in 1983 Garey and Johnson proved the crossing number problem to be NP-Complete and conjectured it is likely to be intractable [2]. Many improved upper and lower bounds have been found for the crossing numbers of the complete and complete bipartite graphs, but no exact solution. [8].

## 2 Crossing Number of ud-ASG

Here we show that computing the crossing number of a given ud-ASG can be done in $O(|E| \log |E|)$ time.

**Theorem 1** *Computing the crossing number of a given ud-ASG can be done in $O(n \log n)$ time*

*Proof.* We propose an $O(|E| \log |E|)$ time algorithm, Cross-ud$(V, E)$, shown in Figure 2. The algorithm essentially accumulate numbers of crossing edges of each edge that proceeds it.

Note that two edges $(x, y)$ and $(u, v)$, in the lexicographic ordering, intersect each other if and

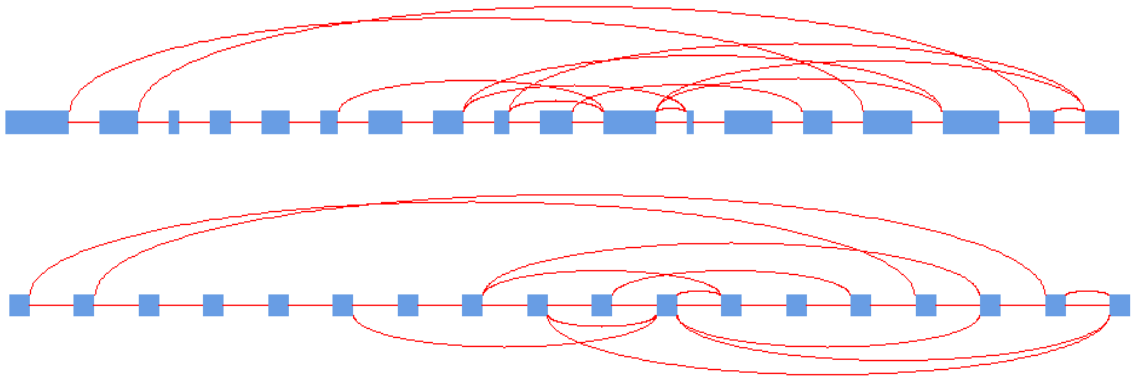

Figure 1: An (upper drawing) alternative splicing graph and its corresponding optimal (two-way) drawing.

only if $x < u < y < v$. It follows that Step 4 of the algorithm correctly discards irrelevant non-intersecting edges $(x, y)$'s since these edges have right endpoints $y \leq u$. It thus implies Step 5 correctly computes the number of intersecting edges $(x, y)$'s for the reason that all these edges $(x, y)$'s must have $x < u < y < v$. Note that the two edges $(u, v)$ and $(u, v')$ with $v < v'$ do not intersect each other since they are visited in the decreasing $v$'s ordering.

Clearly the sorting in Step 1 can be done in $O(|E| \log |E|)$ time. Furthermore, it is easily seen that Step 4 to Step 6 each can be computed in $O(\log |E|)$ time by maintaining some standard priority queue; e.g. a balanced binary tree with augmented data that can be used to compute the RANK() operation for correctly reporting the NUM-PRED$(Q, v)$ function. Since the for loop is executed exactly $O(|E|)$ times, it follows that the algorithm finishes in $O(|E| \log |E|)$ time. □

## 3 Optimal Drawing of ASG

As discussed in previous sections, allowing only upper embedding often leads to too many crossing drawn arcs on the upper plane. The number of crossing arcs can be reduced dramatically by allowing some of these arcs being drawn on the lower plane as illustrated in Figure 1.

Finding efficient ways of drawing alternative splicing graphs with fewest possible crossings is actually a very difficult problem. As an example, it is immediately seen that complete graphs $K_n$ and complete bipartite graphs $K_{n,n}$ and $K_{n,n-1}$ are just a special cases of alternative splicing graphs. Thus an efficient algorithm for optimal drawing of ASG immediately solve the long lasting open problems for computing the crossing numbers of complete graphs and complete bipartite graphs.

Here we propose a branch-and-bound algorithm for optimal drawing of ASG that efficiently finds the drawing with fewest crossings. The algorithm has been tested and implemented to incorporate with our web-based system. We undergo some experiments of the implemented programs and some benchmark data is shown later.

First we describe our branch-and-bound recursive algorithm, ASG-SEARCH$(k, B)$, in Figure 3. The ideas behind the algorithm is as following. First layout the corresponding vertices (exons) on a horizontal line as their natural ordering in the genomic sequence. Now the problem comes to decide whether a considered edge shall be placed in the upper plane or the lower plane. It follows the topological structure is decided once we figure out how to layout the edges in the upper or lower plane. Thus the embedding problem is equivalent to assign a boolean value, 0 or 1, to each edge. That is, consider the searching space $S = \{str[1..|E|] \mid str[i] = 1 \text{ or } 0\}$; i.e., $|S| = 2^{|E|}$. The algorithm essentially needs to find out the optimal element in $S$ whose embedding produces the fewest crossing number.

A brute force approach would need to explore all $2^{|E|}$ configurations and then computing the crossing numbers for each layout resulting an $\Theta(2^{|E|}|E| \log |E|)$ time algorithm. To avoid unnecessary explorations of most inconceivably "bad" configurations, we use the idea of standard branch-and-bound technique with some "cleaver" heuristic searching strategy. In the recursive subroutine ASG-SEARCH$(k, B)$, as shown at Figure 3, the main theme can be viewed as searching a complete binary tree with depth $|E|$. The searching

```
CROSS-UD(V, E)
Input: A set of vertices V, and a set of linking arcs E = {(u, v) | u, v ∈ V}.
Output: The crossing number of the upper drawing alternative splicing graph, G = (V, E).

1 Sort the edges (u, v)'s of E in increasing u (and then decreasing v when u is tight) ordering.
2 k ← 0; Q ← ∅        ▷ Priority Queue.
3 for each edge (u, v) in sorted order do
4     REMOVE-PRED(Q, u)        ▷ Discard vertices at, or on the left of, u.
5     k ← k + NUM-PRED(Q, v)        ▷ crossing arcs.
6     INSERT(Q, v)        ▷ Adding vertex v.
7 return k
```

Figure 2: Computing the crossing number of a ud-ASG in $O(|E| \log |E|)$ time.

```
global Opt[1..|E|], Temp[1..|E|]        ▷ boolean string for drawing configuration of G
       ▷ Temp[i] = 0 if and only if the edge i is drawn at the upper plane.
global BOUND        ▷ global upper bound for the optimal drawing

ASG-SEARCH(k, B)        ▷ drawing the k-th edge of G
Input: k: the k-th edge; B: upper bound of crossings.
1 if k > |E| then        ▷ Terminates.
2     if B < BOUND then
3         BOUND ← B; Opt ← Temp
4     return
5 U ← UP-CROSS(k); L ← LOW-CROSS(k)
6 if U ≤ L then ▷ Greedy heuristic: smaller cost first.
7     if U + B ≥ BOUND then return ▷ Branching bound!
8     Temp[k] ← 0; ASG-SEARCH(k + 1, U + B) ▷ recursive searching
9     if L + B ≥ BOUND then return
10    Temp[k] ← 1; ASG-SEARCH(k + 1, L + B)
11 else ▷ Symmetric cases
12    if L + B ≥ BOUND then return ▷ Branching bound!
13    Temp[k] ← 1; ASG-SEARCH(k + 1, L + B) ▷ recursive searching
14    if U + B ≥ BOUND then return
15    Temp[k] ← 0; ASG-SEARCH(k + 1, U + B)

Main program calls ASG-SEARCH(1, 0) with BOUND ← ∞.
```

Figure 3: A branch-and-bound algorithm for drawing ASG with fewest crossing with heuristic searching.

paths are encoded by the single boolean string $Temp[1..|E|]$. Note that a completely 0-1 filled string $Temp[]$ corresponds to a leaf node of the imaginary search tree. Thus finding the optimal configuration is equivalent to finding a leaf node with the minimum crossing number. Furthermore, once some values of some leaf nodes have been found, their minimum crossing number can be viewed as a bounded value for other unexploited paths. The currently obtained minimum crossing configuration is stored at the boolean array $Opt[]$ with the minimum crossing stored at the global variable $BOUND$.

It follows that, whenever a prefix path of some unfinished paths possess a prefix crossing number being greater than $BOUND$, the remaining paths (corresponding to some subset of the searching space $S$) do not need to be further exploited. Here the spirit of branching and bound is applied. It is easily seen that the Step 3 of the searching subroutine ASG-Search$(k, B)$ set up the currently branching bound, $BOUND$. Also note that Steps 7, 9, 12, and 14 of ASG-Search are where the bounded branching take places. Most of the unneeded computation efforts are saved there.

Finally, for the branch-and-bound searching algorithm to be really efficient, it will be beneficial if we can reach some "good" answers as soon as possible. The idea is to find a relatively low value of the bounded valued, $BOUND$, as soon as possible. Thus much unwanted computation can be avoided. To do so, we employ the heuristic searching strategy at Step 5 and Step 6 of ASG-Search. Note that Step 5 computes Up-Cross$(k)$ and Low-Cross$(k)$, these are the crossing number caused by $k$-th edge when it is placed at the upper plane and lower plane. The algorithm then further searching downward which one comes smaller. The idea behind these two lines is the greedy approach. Whenever the $k$-th edge is in consideration, it would be wiser to place it in the (upper or lower) plane where it cause less cost.

## 4 Experimental Results

The resulting branch and bound searching routine is rather efficient. For example, in our implementation of the algorithm in $C$, that is run under the Red Hat Linux 7.2 system with experimental machine equipped with Pentium-4 CPU for 1000Mhz. Just as an example of showing the efficiency of the heuristic branch and bound algorithm, we note that the program correctly finds the embedding of complete graph $K_9$ in 1 seconds; and the program correctly finds the optimal embedding of complete graph $K_{10}$ in 83 seconds. The resulting crossing numbers $\nu(K_9) = 36, \nu(K_{10}) = 60$ are correctly reported by the program. In contrast, note that $K_9$ possess 36 edges and $K_{10}$ has 45 edges; that is, the searching spaces have sizes $2^{36} = 68,719,476,736, 2^{45} = 35,184,372,088,832$. It is estimated that the program explores about 1/1000 over the whole searching space; the computation speed up is about 1,000 times faster comparing to the brute force approach in these cases. Here we summarize our experiments for searching the optimal embedding of some complete graphs complete bipartite graphs in Table 1.

Most of the program for finding the optimal embedding of the alternative splicing graphs have been incorporated with our quantitative alternative splicing analysis system web site at `http://bioinfo.cs.pu.eud.tw/ASF/dasg` [1]. The system with easy internet accessibility is mostly built by using the conventional PHP system, that has a long reputation of being easily integrated with HTML and CGI programs; the optimal embedding program is one of these CGI programs and have been equipped with PHP programs for drawing the resulted optimal embedded ASG for visualization.

## 5 Concluding Remarks

Due to all these important aspects of genomic alternative splicing mechanism that produces different mRNA's, biologists require means of analyzing and visualizations of the alternative splicing forms. In this paper, we consider ways of drawing a given alternative splicing graph on the plane.

Given a graph $G = (V, E)$, here in this paper we show that computing the crossing number of a given upper drawing ASG can be done in $O(|E| \log |E|)$ time.

The optimal (fewest crossing) drawing alternative splicing graphs is related to the long known difficult problem of crossing number problem [3, 2, 8]. Here we propose an efficient branch-and-bound recursive algorithm with heuristic searching strategy. Some experimental results are also discussed in the paper.

| $G$ | $n$ | $m$ | $\nu(G)$ | $T$ | $G$ | $n$ | $m$ | $\nu(G)$ | $T$ |
|---|---|---|---|---|---|---|---|---|---|
| $K_3$ | 3 | 3 | 0 | $< 0.01$ | | | | | |
| $K_4$ | 4 | 6 | 0 | $< 0.01$ | | | | | |
| $K_5$ | 5 | 10 | 1 | $< 0.01$ | $K_{2,2}$ | 4 | 4 | 0 | $< 0.01$ |
| $K_6$ | 6 | 15 | 3 | $< 0.01$ | $K_{3,3}$ | 6 | 9 | 1 | $< 0.01$ |
| $K_7$ | 7 | 21 | 9 | $< 0.01$ | $K_{4,4}$ | 8 | 16 | 4 | $< 0.01$ |
| $K_8$ | 8 | 28 | 18 | $< 0.01$ | $K_{5,5}$ | 10 | 25 | 16 | $< 0.01$ |
| $K_9$ | 9 | 36 | 36 | 1.08 | $K_{6,6}$ | 12 | 36 | 36 | 0.265 |
| $K_{10}$ | 10 | 45 | 60 | 82.4 | $K_{7,7}$ | 14 | 49 | 81 | 261.5 |
| $K_{11}$ | 11 | 55 | 100 | 2332 | $K_{8,8}$ | 16 | 64 | 144 | 261,979 |

Name: $G$: graph; $n$: number of nodes; $m$: number of edges;
$\nu(G)$: crossing number; $T$:CPU time in seconds).

Table 1: Searching for the optimal embedding of some complete graphs.


## Acknowledgements

The author would like to thank my graduate students Po-Shun Yu, Hsun-Chang Chang, and Tze-Wei Huang for their assistance of building up the Web system, and we thank many instructive discussions and information concerning the biological processes of alternative splicings within the Providence university Bioinfomatic forum [1].


## References


[1] Providence University Bioinfo Forum. http://bioinfo.cs.pu.edu.tw.

[2] M. R. Garey and D. S. Johnson. Crossing number is NP-complete. *SIAM Journal of Algebraic and Discrete Methods,*, 4:312–316, 1983.

[3] Richard K. Guy. Crossing numbers of graphs. In *Graph Theory and Applications*, Lecture Notes in Mathematics 303, pages 111–124. Springer-Verlag, 1972.

[4] Nathalie Heuze, Sophie Olayat, Ninette Gutman, Marie-Louise Zani, and Yves Courty. Molecular cloning and expression of an alternative hklk3 transcript coding for a variant protein of prostate-specific antigen. *Cancer Res.*, 59:2820–2824, 1999.

[5] Zhengyan Kan, Eric C. Rouchka, Warren R. Gish, and David J. Gene structure prediction and alternative splicing analysis using genomically aligned ests. *Genome Res.*, 11:889–900, 2001.

[6] Zhengyan Kan, David States, and Warren Gish. Selecting for functional alternative splices in ests. *Genome Res.*, 12:1837–1845, 2002.

[7] D. J. Kleitman. The crossing number of $K_{5,n}$. *Journal of Combinatorial Theory*, 9:315–323, 1971.

[8] Annegret Liebers. Planarizing graphs - a survey and annotated bibliography. *Journal of Graph Algorithms and Applications*, 5(1):1–74, 2001.

[9] A. J. Lopez. Alternative splicing of pre-mrna: developmental consequences and mechanisms of regulation. *Annu. Rev. Genet.*, 32:279–305, 1998.

[10] Qiang Xu, Barmak Modrek, and Christopher Lee. Genome-wide detection of tissue-specific alternative splicing in the human transcriptome. *Nucleic Acids Res.*, 30:3754–3766, 2002.

[11] K. Zarankiewicz. On a problem of P. Tur'an concerning graphs. *Fund. Math.*, 41:137–145, 1954.